

Documento Técnico de Arquitectura

WanderSync Travel Solutions: plataforma de empaquetamiento turístico dinámico (vuelo + hotel + auto) con microservicios, API Gateway GraphQL, patrón SAGA e ingesta distribuida con Dask y Prefect.

Incluye diagramas de arquitectura y de secuencia SAGA (éxito y compensación)

Docker Compose

GraphQL · Strawberry

SAGA orquestada

Dask

Prefect

FastAPI

PostgreSQL 16

React 19

Contenido

1. Resumen

2. Contexto y problema

3. Trazabilidad: requisitos → solución

4. Vista general de la arquitectura

4.1 Responsabilidades

4.2 Estilo arquitectónico

4.3 Diagrama de componentes y despliegue

5. Decisiones arquitectónicas

5.1 Una base de datos por microservicio

5.2 SAGA por orquestación (no coreografía)

5.3 Idempotencia con `saga_id`

5.4 Compensar es cambiar estado, no borrar

5.5 GraphQL en un Gateway propio (Strawberry)

5.6 Persistencia e integración con GraphQL

5.7 `NUMERIC` para dinero

5.8 Solo el frontend y el Gateway exponen puertos

6. Patrón SAGA

6.1 Pasos y compensaciones

6.2 Máquina de estados

6.3 Diagrama de secuencia — camino exitoso (happy path)

6.4 Diagrama de secuencia — fallo y compensaciones

6.5 Diagrama de secuencia — compensación que no se logra

6.5b Diagrama de secuencia — cancelación pedida por el cliente

6.6 Evidencia obtenida en las pruebas

7. API Gateway GraphQL

7.1 Esquema (SDL)

7.2 Cómo se elimina el over-fetching

7.3 Cálculo del precio

7.4 Destinos

8. Ingesta distribuida: Dask + Prefect

8.1 Roles

8.2 Pipeline por ciudad

8.3 Política de reintentos

8.4 Fuente de datos y scraping responsable

8.5 Segunda fuente: Trivago (BeautifulSoup) con respaldo

8.5b Tercera fuente: ofertas de Booking.com

8.6 Cómo se evita el cuello de botella en la persistencia

9. Ciberseguridad por diseño

9.1 Almacenamiento de contraseñas: Argon2id

9.2 Sesiones y mitigación de Session Fixation

9.3 Rate Limiting

9.3b Control de acceso a las reservas (IDOR)

9.4 Endurecimiento del frontend

9.5 Seguridad de la cadena de suministro

10. Frontend

11. Despliegue con Docker Compose

11.1 URL pública (Cloudflare Tunnel)

12. Guion de la demostración

13. Limitaciones conocidas y trabajo futuro

14. Conclusiones

1. Resumen

WanderSync vende paquetes que combinan un vuelo, un hotel y un auto. Cada componente lo gestiona un microservicio con **su propia base de datos**, así que reservar un paquete es una **transacción distribuida**: no existe un `COMMIT` que abarque los tres servicios.

La solución se apoya en cinco piezas:

Pieza	Rol en la arquitectura
Patrón SAGA (orquestación)	El servicio Órdenes ejecuta vuelo → hotel → auto y, si un paso falla, compensa en orden inverso. Elimina las "reservas huérfanas".
API Gateway GraphQL	Única puerta de entrada del frontend. Consolida los tres servicios en una consulta y evita el over-fetching.
Dask	Clúster (scheduler + 2 workers) que paraleliza el scraping, la limpieza y la ingesta de tarifas reales.
Prefect	Orquesta el flujo de ingesta sobre Dask: programación cada 30 min, reintentos con backoff y panel de observabilidad.
Docker Compose	Todo el ecosistema (13 contenedores, 14 con el túnel opcional) se levanta con <code>docker compose up --build</code> .

Stack: **Python 3.12** (FastAPI, Strawberry GraphQL, psycpg 3), **PostgreSQL 16**, **React 19 + Vite + Tailwind** servido por **nginx**.

2. Contexto y problema

La arquitectura previa de WanderSync tenía dos fallas:

1. **Reservas huérfanas.** El pago y el vuelo se confirmaban, pero si la reserva del hotel o del auto fallaba en la red, la orden quedaba en un estado indeterminado y nada revertía lo ya reservado.
2. **Cuellos de botella en la sincronización de tarifas.** La actualización masiva de precios bloqueaba la capa de persistencia y la red, afectando a los servicios que atienden clientes.

Además, la empresa quiere dejar de depender de APIs comerciales cerradas y obtener tarifas mediante **extracción automatizada** de fuentes públicas.

Objetivos de diseño que se derivan:

- **Consistencia eventual garantizada:** una reserva termina siempre en *todo confirmado* o *todo cancelado*, nunca a medias.
- **Ingesta desacoplada:** el scraping corre en segundo plano, distribuido, y escribe en lote; nunca en el camino de una petición del cliente.
- **Observabilidad:** se puede ver en qué estado está cada SAGA y cada ejecución de ingesta.
- **Seguridad por diseño:** autenticación robusta, protección contra fuerza bruta y control de la cadena de suministro.

3. Trazabilidad: requisitos → solución

Requisito del enunciado	Cómo se cumple	Dónde está
Microservicios Vuelos, Hoteles, Autos, Órdenes y Gateway	Cinco servicios FastAPI independientes, cada uno con su <code>Dockerfile</code> , más el servicio Ofertas	<code>vuelos/</code> , <code>hoteles/</code> , <code>autos/</code> , <code>ordenes/</code> , <code>gateway/</code> , <code>ofertas/</code>
Despliegue con un único comando	<code>docker compose up --build</code> levanta los 13 contenedores, con <code>depends_on</code> + healthchecks	<code>docker-compose.yml</code>
GraphQL como API Gateway unificado	Strawberry sobre FastAPI; el frontend solo conoce <code>/graphql</code>	<code>gateway/main.py</code> , <code>frontend/nginx.conf</code>
Consultas complejas sin over-fetching	<code>paquetes(destino, noches, personas)</code> consolida 3 servicios en paralelo; el cliente elige los campos	\$7
Mutaciones de reserva	<code>reservarPaquete</code> dispara la SAGA	\$6, \$7
SAGA con happy path y compensaciones automáticas	Orquestador con compensación en orden inverso, reintentos e idempotencia	<code>ordenes/main.py</code> , \$6
Fallo simulado	Interruptor <code>FORZAR_FALLO_AUTOS=true</code> (Autos responde 503)	<code>autos/main.py</code>
Dask para recolección e ingesta	Scheduler + 2 workers; <code>DaskTaskRunner</code> reparte las tareas	<code>ingesta/flow.py</code> , \$8
Fuentes reales (Google Flights, Kayak, Booking...)	Hostelworld en vivo; Trivago con BeautifulSoup (bloquea al bot, se usan páginas guardadas de respaldo); ofertas de Booking por categoría desde páginas guardadas	<code>ingesta/flow.py</code> , <code>ingesta/trivago.py</code> , \$8.4-8.5
Prefect con retries y monitoreo	Flow <code>ingesta-hoteles</code> , retries 5 s/15 s/45 s, panel en <code>:4200</code>	<code>ingesta/flow.py</code> , \$8
Persistencia con integración GraphQL	PostgreSQL expuesto mediante el Gateway GraphQL (justificación en \$5.6)	<code>db/init.sql</code> , \$5.6
Session Fixation	Se destruye la sesión previa y se emite un ID nuevo en cada login	<code>gateway/seguridad.py</code> , \$9.2
Hashing robusto	Argon2id (t=3, m=64 MiB, p=4) con sal aleatoria	\$9.1
Rate Limiting en login, pago y checkout	Ventana deslizante por operación; HTTP 429	\$9.3
Auditoría de la cadena de suministro	<code>pip-audit</code> (6 servicios) y <code>npm audit</code> (frontend): 0 vulnerabilidades	<code>seguridad/</code> , \$9.5
Frontend que consume la API	SPA React con panel SAGA e inspector GraphQL	<code>frontend/</code> , \$10

4. Vista general de la arquitectura

4.1 Responsabilidades

Componente	Responsabilidad	Datos que posee
Frontend (nginx)	Interfaz; reenvía <code>/graphql</code> al Gateway; cabeceras de seguridad	—
Gateway	Esquema GraphQL, composición de paquetes, autenticación, sesiones, rate limiting	BD <code>auth</code> (<code>usuarios</code> , <code>sesiones</code>)
Vuelos	Catálogo de vuelos (semilla + Booking); acción y compensación de reserva de vuelo	BD <code>vuelos</code>
Hoteles	Catálogo de hoteles (semilla + scraping); reserva/cancelación de hotel	BD <code>hoteles</code>
Autos	Catálogo de autos (semilla + precio medio de Booking); reserva/cancelación de auto; interruptor de fallo	BD <code>autos</code>
Órdenes	Orquestador de la SAGA; bitácora de cada transacción; reservas por usuario y cancelación	BD <code>ordenes</code> (<code>sagas</code>)
Ofertas	Sirve las ofertas de Booking por categoría (las escribe la ingesta)	BD <code>ofertas</code>
Túnel (opcional)	Publica solo el frontend en una URL <code>https://</code> con Cloudflare Tunnel	—
Prefect server	Historial de flows y tareas, programación, panel web	Interna de Prefect
Dask scheduler / workers	Reparto y ejecución distribuida de las tareas de ingesta	—
Ingesta	Registra el flow en Prefect (<code>serve</code>) y lo dispara cada 30 min	—

4.2 Estilo arquitectónico

- **Microservicios con *database per service***: cada servicio es dueño exclusivo de sus datos.
- **API Gateway** como fachada única (patrón *Backend for Frontend* simplificado).
- **SAGA orquestada** para la consistencia entre servicios.
- **Procesamiento por lotes asíncrono** (pipeline ETL distribuido) para la ingesta, separado del camino de las peticiones.

4.3 Diagrama de componentes y despliegue

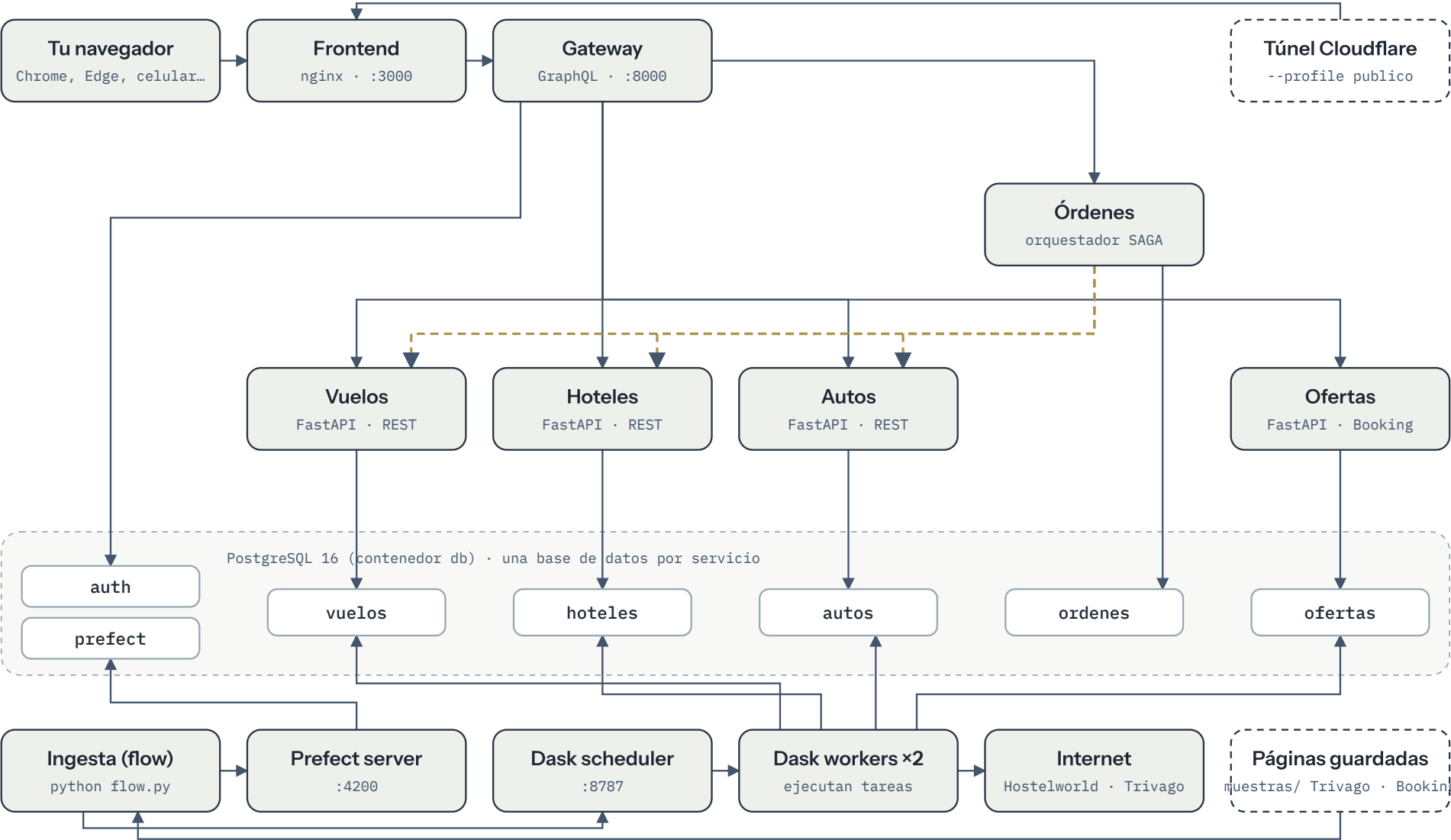


Figura 1. Diagrama de componentes y despliegue. Línea continua: petición HTTP o conexión a la BD. Línea discontinua dorada: pasos de la SAGA (Órdenes llama a los participantes). Caja punteada: elemento opcional o carpeta, no contenedor. Solo publican puerto: Frontend :3000, Gateway :8000, Prefect :4200 y Dask :8787.

5. Decisiones arquitectónicas

Cada decisión se presenta como un registro breve (contexto → decisión → consecuencias).

5.1 Una base de datos por microservicio

- **Contexto.** Si los servicios compartieran tablas, un cambio en una rompería a los demás y el sistema seguiría siendo un monolito distribuido.
- **Decisión.** Un único servidor PostgreSQL (por simplicidad de despliegue) con **una base de datos lógica por servicio**: `vuelos`, `hoteles`, `autos`, `ordenes`, `auth` (Gateway) y `ofertas`, más la de Prefect. Cada servicio recibe solo la URL de la suya por variable de entorno.
- **Evolución del esquema.** `db/init.sql` solo corre al crear el volumen; por eso Vuelos, Autos y Órdenes aplican al arrancar migraciones idempotentes (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS`) y Ofertas crea su base y su tabla si no existen. Una BD creada antes recibe las columnas nuevas sin borrar datos.
- **Consecuencias.** (+) Aislamiento y autonomía. (–) Ya no hay JOIN ni transacciones entre servicios → **por eso es necesario el patrón SAGA**. En producción, cada BD podría moverse a su propio servidor sin cambiar código.

5.2 SAGA por orquestación (no coreografía)

Criterio	Orquestación (elegida)	Coreografía
Dónde vive la lógica del flujo	En un solo servicio (Órdenes)	Repartida entre todos los servicios
Infraestructura adicional	Ninguna (HTTP)	Broker de mensajes (Kafka, RabbitMQ...)
Depuración y trazabilidad	Una bitácora central (<code>sagas</code>)	Hay que reconstruir el flujo desde eventos
Acoplamiento	Órdenes conoce a los participantes	Los servicios se conocen solo por eventos
Riesgo de dependencias cíclicas	Bajo	Alto con muchos participantes

- **Decisión.** Orquestación: con tres participantes y un flujo lineal, centralizar la lógica hace el sistema más fácil de entender, demostrar y dibujar, y evita operar un broker.
- **Consecuencias.** Órdenes es un punto central; se mitiga con la bitácora persistente (§13 describe la recuperación pendiente).

5.3 Idempotencia con `saga_id`

- El orquestador genera un `UUID` por transacción y lo envía a cada participante.
- En cada tabla `reservas`, `saga_id` es **llave primaria** e `INSERT ... ON CONFLICT (saga_id) DO NOTHING`.
- La cancelación es un `UPDATE ... SET estado = 'CANCELADA'`, idempotente por naturaleza.
- **Consecuencia clave:** reintentar cualquier paso o compensación es seguro. Esto permite reintentar con backoff y compensar también el paso que dio *timeout* (puede haber reservado sin que el orquestador se enterara).

5.4 Compensar es cambiar estado, no borrar

Las reservas canceladas permanecen con `estado = 'CANCELADA'`. Así queda trazabilidad completa para auditoría y para la demo.

5.5 GraphQL en un Gateway propio (Strawberry)

- **Contexto.** El frontend necesita combinar datos de tres servicios; con REST serían tres llamadas y datos de sobra.
- **Decisión.** Gateway GraphQL en Python (Strawberry + FastAPI) que consulta los servicios **en paralelo** (`asyncio.gather`) y compone el tipo `Paquete`.
- **Consecuencias.** (+) Una sola petición, campos a elección del cliente, un único lugar para aplicar seguridad. (-) El Gateway conoce las direcciones internas de los servicios.

5.6 Persistencia e integración con GraphQL

El enunciado sugiere un motor con GraphQL nativo (por ejemplo, Supabase con `pg_graphql`). Se evaluó y se optó por **PostgreSQL expuesto a través del Gateway GraphQL**:

- `pg_graphql` genera un esquema GraphQL **de una sola base de datos**. Usarlo como API del frontend lo acoplaría directamente a las tablas y rompería *database per service*: el paquete combina datos de cuatro bases distintas.
- Exponer la BD directamente al cliente se saltaría la capa de autenticación, rate limiting y la SAGA.
- Con el Gateway, la fuente unificada para el frontend sigue siendo un esquema GraphQL, pero respaldado por servicios que validan las reglas de negocio.
- Los datos que procesan los workers de Dask se persisten directamente en la BD `hoteles` y aparecen de inmediato en la consulta `paquetes` (campo `fuelle: "hostelworld"`).

Migrar a Supabase sería directo: es PostgreSQL, así que bastaría cambiar las `DATABASE_URL`.

5.7 `NUMERIC` para dinero

Los precios se almacenan como `NUMERIC(12,2)` y el scraper limpia con `Decimal`: `FLOAT` introduce errores de redondeo binario (`0.1 + 0.2 ≠ 0.3`) que, con dinero, descuadran cuentas. (Ver la limitación sobre el transporte en §13.)

5.8 Solo el frontend y el Gateway exponen puertos

Vuelos, Hoteles, Autos y Órdenes no publican puertos: solo son accesibles dentro de la red de Docker, así que **nadie puede saltarse el Gateway**. El frontend llega al Gateway a través de nginx en el mismo origen (`localhost:3000/graphql`). El puerto 8000 del Gateway se mantiene abierto para la demostración con GraphiQL.

6. Patrón SAGA

6.1 Pasos y compensaciones

#	Participante	Acción (<code>POST /reservas</code>)	Compensación (<code>POST /reservas/{saga_id}/cancelar</code>)
1	Vuelos	Inserta reserva <code>CONFIRMADA</code>	Estado → <code>CANCELADA</code>
2	Hoteles	Inserta reserva <code>CONFIRMADA</code>	Estado → <code>CANCELADA</code>
3	Autos	Inserta reserva <code>CONFIRMADA</code> (o 503 si <code>FORZAR_FALLO_AUTOS</code>)	Estado → <code>CANCELADA</code>

El paso 3 solo existe si el paquete trae auto: donde Booking no tiene alquiler de autos (Pereira, San Andrés, Cancún, Lima...) y el paquete es vuelo + hotel (`auto_id` nulo), así que la SAGA tiene dos pasos.

Reglas del orquestador (`ordenes/main.py`):

1. Registra la SAGA en la tabla `sagas` con estado `EN_CURSO` **antes** de empezar.
2. Ejecuta los pasos **en orden** y actualiza la bitácora en cada uno.
3. Apila cada participante **antes** de llamarlo. Si la llamada falla o da *timeout*, también se compensa (cancelar algo que no se reservó es inofensivo gracias a la idempotencia).
4. Ante un fallo, compensa en **orden inverso** (lo último primero).
5. Cada compensación se reintenta con **backoff exponencial** (1 s, 2 s, 4 s, 8 s, 16 s): una compensación no puede rendirse.
6. Si una compensación agota sus reintentos, la SAGA queda en `REQUIERE_ATENCION`: nunca falla en silencio.
7. Cada SAGA guarda su dueño (`usuario_id`) y un resumen de lo reservado (destino, vuelo, hotel, auto, noches, personas y total). Lo envía el Gateway, porque Órdenes no puede leer las bases de datos de los otros servicios.

Cancelación por el cliente. `POST /ordenes/{id}/cancelar` reutiliza las mismas compensaciones: primero cambia el estado de `CONFIRMADA` a `CANCELANDO` con un único `UPDATE ... WHERE estado = 'CONFIRMADA' AND usuario_id = ...` (transición atómica: si llegan dos cancelaciones a la vez, solo una la encuentra en `CONFIRMADA` y la otra recibe 409), y después cancela auto → hotel → vuelo con los mismos reintentos e idempotencia. Termina en `CANCELADA`, o en `REQUIERE_ATENCION` si alguna cancelación no se logró.

6.2 Máquina de estados

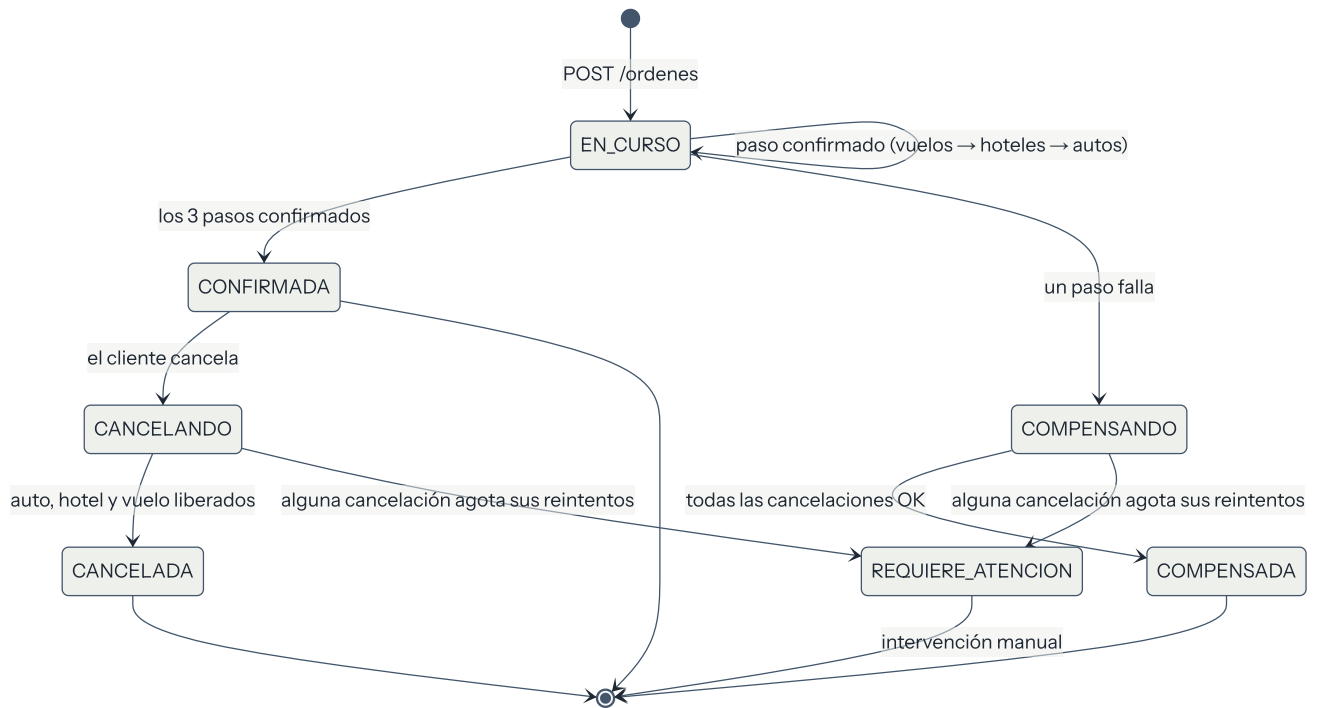


Figura 2. Máquina de estados

6.3 Diagrama de secuencia — camino exitoso (happy path)

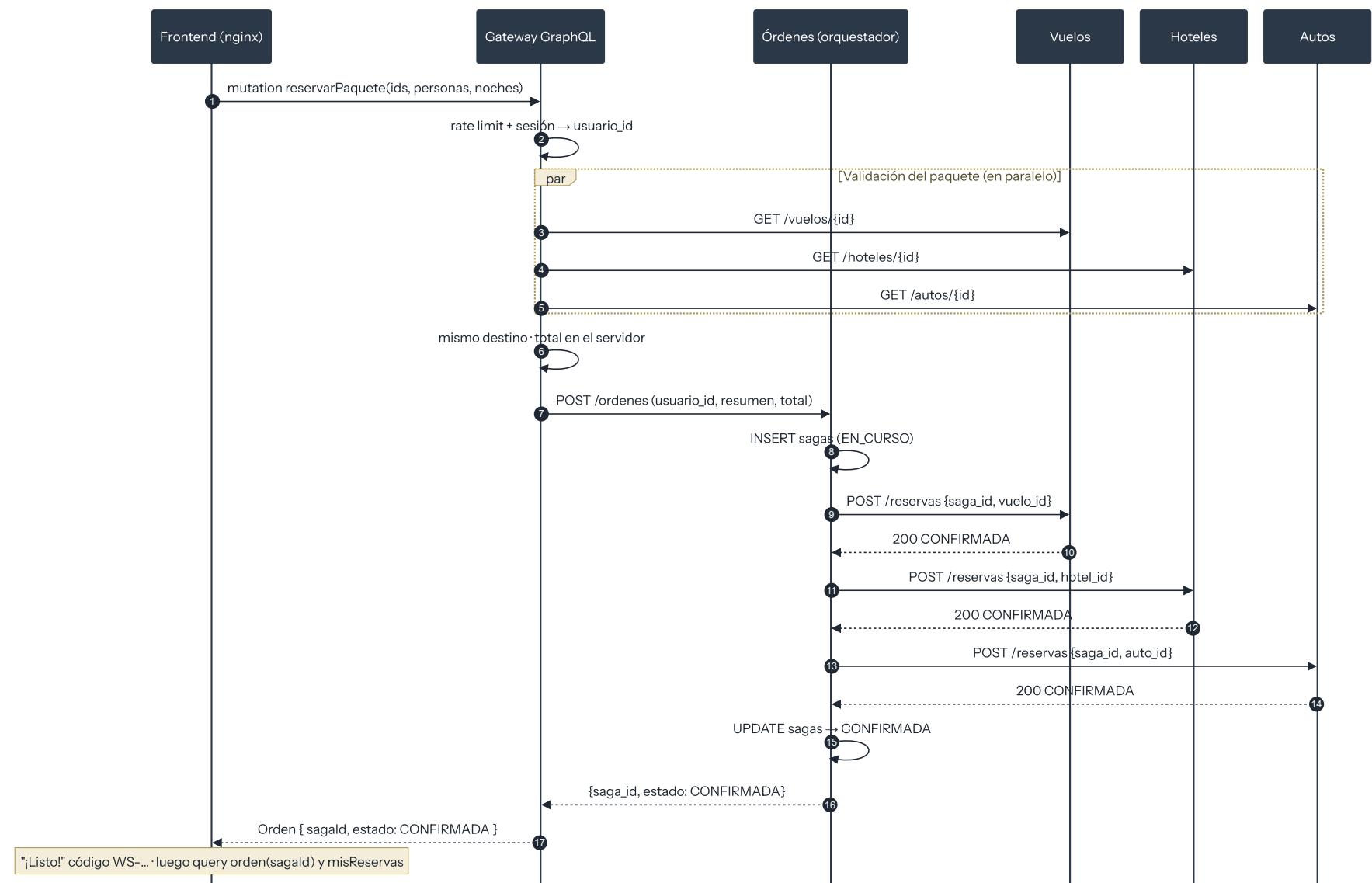


Figura 3. Diagrama de secuencia — camino exitoso (happy path)

6.4 Diagrama de secuencia — fallo y compensaciones

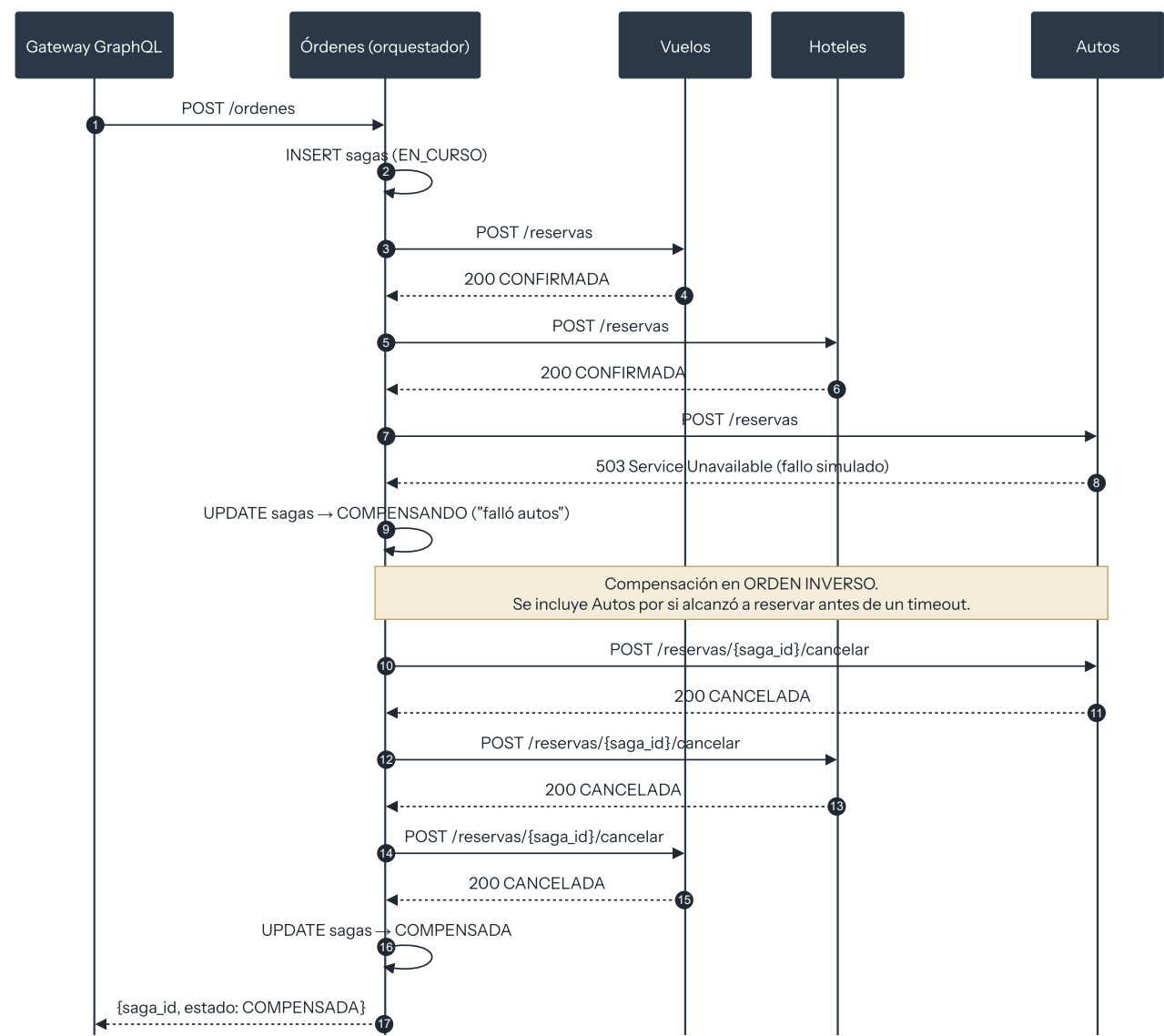


Figura 4. Diagrama de secuencia — fallo y compensaciones

6.5 Diagrama de secuencia — compensación que no se logra

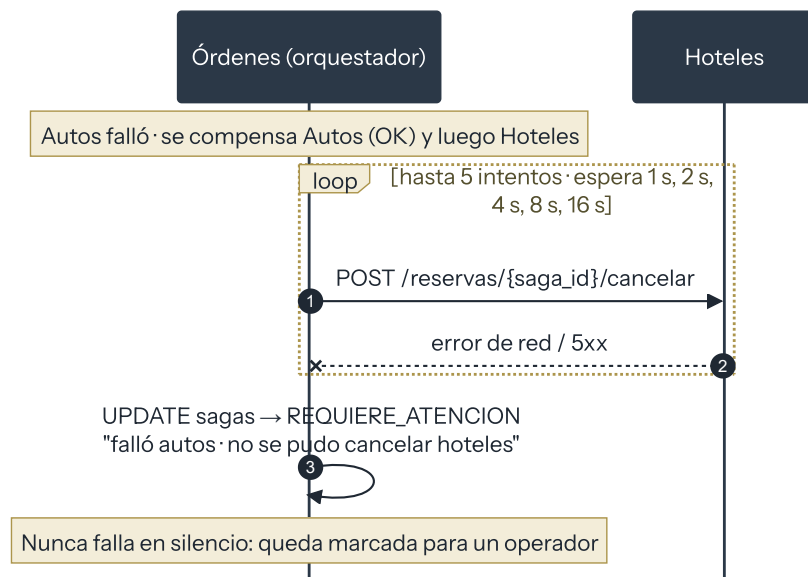


Figura 5. Diagrama de secuencia — compensación que no se logra

6.5b Diagrama de secuencia — cancelación pedida por el cliente

La cancelación desde "Mis reservas" reutiliza las compensaciones de la SAGA sobre una reserva ya confirmada.

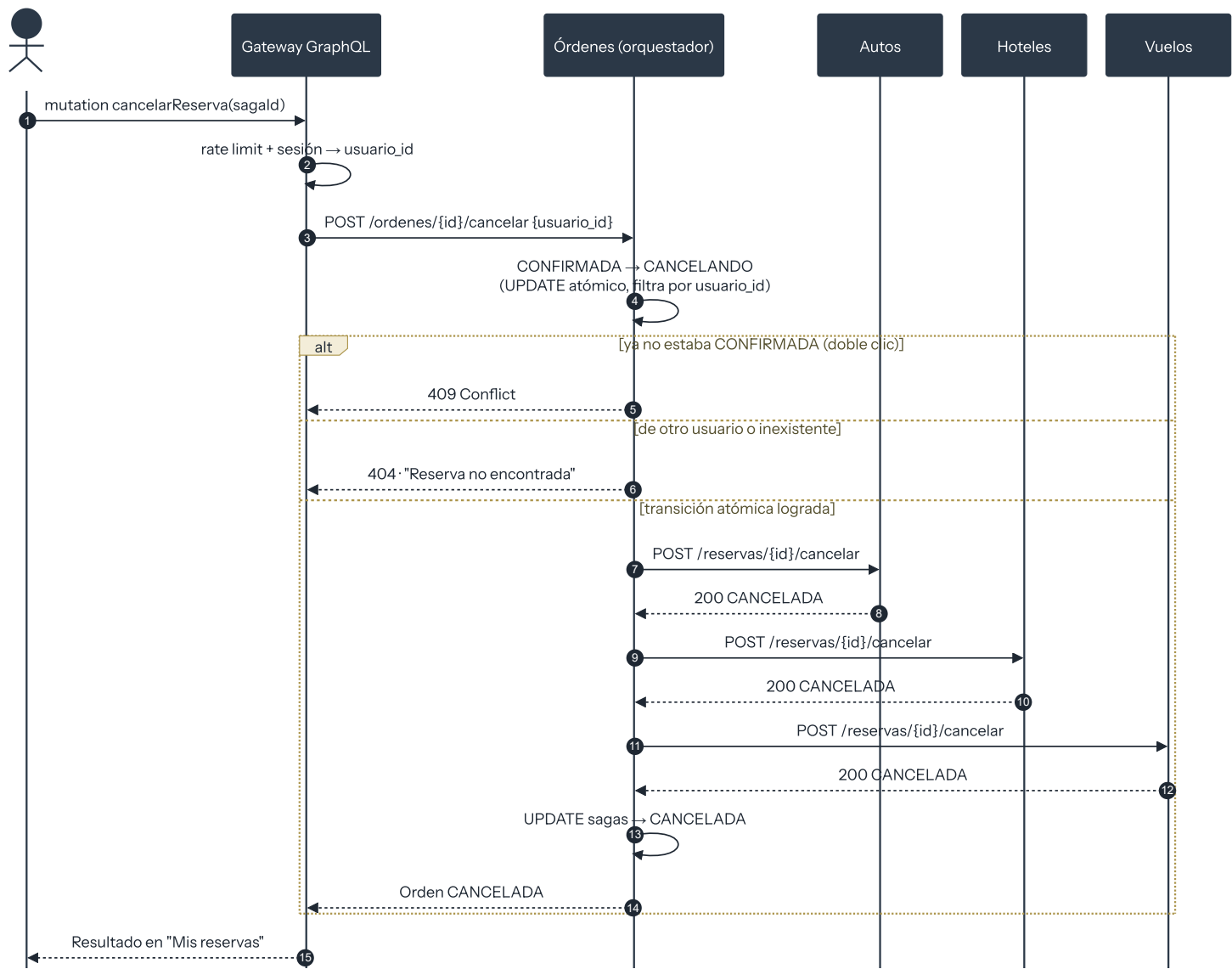


Figura 6. Diagrama de secuencia — cancelación pedida por el cliente

6.6 Evidencia obtenida en las pruebas

Log del orquestador con `FORZAR_FALLO_AUTOS=true`:

```
[SAGA 4c6df63d-...] EN_CURSO: reservando vuelos
[SAGA 4c6df63d-...] EN_CURSO: reservando hoteles
[SAGA 4c6df63d-...] EN_CURSO: reservando autos
[SAGA 4c6df63d-...] COMPENSANDO: falló autos: Server error '503 Service Unavailable' for url
'http://autos:8000/reservas'
[SAGA 4c6df63d-...] compensado: autos
[SAGA 4c6df63d-...] compensado: hoteles
[SAGA 4c6df63d-...] compensado: vuelos
[SAGA 4c6df63d-...] COMPENSADA: falló autos; reservas previas canceladas
```

Estado en la BD de hoteles tras una reserva exitosa y una compensada:

```
estado | count
-----+-----
CANCELADA |      1
CONFIRMADA|      1
```

7. API Gateway GraphQL

7.1 Esquema (SDL)

```
type Query {
  "Paquetes disponibles (vuelo + hotel + auto) hacia una ciudad destino"
  paquetes(destino: String!, noches: Int! = 1, personas: Int! = 1): [Paquete!]! # personas: 1 a
9
  "Ciudades con paquetes hoy: tienen vuelo Y alojamiento"
  destinos: [Destino!]!
  "Estado y resumen de una reserva propia (requiere sesión; la de otro usuario devuelve null)"
  orden(sagaId: String!): Orden
  "Reservas del usuario de la sesión, la más reciente primero (requiere sesión)"
  misReservas: [Orden!]!
  "Ofertas de Booking por categoría (alojamiento, vuelos, coches, atracciones)"
  ofertas(categoria: String): [Oferta!]!
  "Usuario de la sesión actual (null si no hay sesión)"
  yo: Usuario
}

type Mutation {
  registrar(email: String!, password: String!): Usuario!
  iniciarSesion(email: String!, password: String!): Usuario! # emite una sesión NUEVA
  cerrarSesion: Boolean!
  # requiere sesión; el Gateway valida el paquete y calcula el total (el cliente no manda precios)
  reservarPaquete(vueloId: Int!, hotelId: Int!, autoId: Int, personas: Int! = 1, noches: Int! =
1): Orden!
  cancelarReserva(sagaId: String!): Orden! # requiere sesión; solo reservas propias en estado
CONFIRMADA
}

type Paquete { vuelo: Vuelo! hotel: Hotel! auto: Auto personas: Int! habitaciones: Int!
autos: Int! precioTotal: Float! } # auto null = vuelo + hotel
type Vuelo { id: Int! origen: String! destino: String! precio: Float! fuente: String! }
type Hotel { id: Int! nombre: String! ciudad: String! precioNoche: Float! rating: Float
fuente: String! }
type Auto { id: Int! modelo: String! ciudad: String! precioDia: Float! fuente: String! }
type Destino { codigo: String! nombre: String! pais: String vueloDesde: Float! }
type Orden { sagaId: String! estado: String! paso: String personas: Int noches: Int
destino: String
vuelo: String hotel: String auto: String total: Float creado: String }
type Oferta { categoria: String! destino: String! titulo: String detalle: String ciudad:
String cantidad: Int precioDesde: Float! unidad: String campana: String url: String fuente:
String! }
type Usuario { id: Int! email: String! }
```

7.2 Cómo se elimina el over-fetching

- **En el borde (cliente ↔ Gateway):** el cliente declara los campos. La tarjeta de paquete del frontend no pide `hotel.ciudad` ni `auto.ciudad` (ya conoce el destino), así que no viajan. El inspector GraphQL de la interfaz muestra la consulta exacta y los bytes recibidos.
- **En el interior (Gateway ↔ servicios):** cada servicio filtra por ciudad **en su propia base de datos** (`WHERE destino = %s`); entre servicios no viajan registros que no se van a usar.
- **Una sola ida y vuelta:** las tres consultas internas se lanzan en paralelo con `asyncio.gather`; la latencia es la del servicio más lento, no la suma.

Ejemplo de consulta mínima y respuesta:

```
{ paquetes(destino: "MDE", noches: 3) { precioTotal hotel { nombre } } }
```

```
{ "data": { "paquetes": [ { "precioTotal": 751630.0, "hotel": { "nombre": "Casa Kiwi" } } ] } }
```

7.3 Cálculo del precio

```
precioTotal = vuelo.precio × personas + (hotel.precioNoche × habitaciones + auto.precioDia × autos) × noches
```

con `habitaciones = [personas / 2]` (hasta 2 personas por habitación) y `autos = [personas / 5]` (hasta 5 por vehículo; 0 si el paquete no trae auto). El Gateway valida 1 a 9 personas y 1 a 30 noches; el servicio Órdenes vuelve a validar el rango (Pydantic).

La misma función calcula el precio al **listar** y al **reservar**. Al reservar, el navegador solo envía ids, personas y noches: el Gateway pide vuelo, hotel y auto por id a cada servicio (en paralelo), comprueba que sean del mismo destino y calcula el total, que queda guardado en la SAGA. Modificar la petición no cambia lo que se cobra.

7.4 Destinos

Código	Ciudad	Alojamiento	Vuelo BOG →	Auto
MDE·CTG·SMR	Medellín, Cartagena, Santa Marta	Hostelworld + Trivago	Booking + semilla	Booking + semilla
CLO·BAQ·BGA	Cali, Barranquilla, Bucaramanga	Hostelworld	Booking	Booking (precio medio por día)
PEI·ADZ	Pereira, San Andrés	Hostelworld	Booking	— (paquete vuelo + hotel)
MAD	Madrid	Hostelworld	Booking	Booking

Ampliación a 50 vuelos. La página "Vuelos desde Bogotá" pinta 10 tarjetas, pero su estado inicial (`window.__INITIAL_STATE__.flyAnywhere.results`) trae **50 destinos** con código IATA, país y precio por persona; el parser lee ese JSON. Todos pasan al catálogo de Vuelos. El flow descarga Hostelworld para **30** de ellos (14 en Colombia, 10 en América y 6 en Europa: Lima, Quito, Buenos Aires, Panamá, Ciudad de México, Cancún, Miami, Nueva York, Orlando, Los Ángeles, Madrid, Barcelona, París, Roma, Londres y Ámsterdam), así que hasta 30 destinos tienen paquete. La tabla anterior resume los 9 primeros.

La query `destinos` cruza los destinos con vuelo (`GET /destinos` de Vuelos, con nombre y país) y las ciudades con alojamiento (`GET /ciudades` de Hoteles): solo esos tienen paquete. Los demás (Cúcuta, Montería, Valledupar, Caracas, Fráncfort...) aparecen en el selector de destino con sus ofertas de Booking.

8. Ingesta distribuida: Dask + Prefect

8.1 Roles

- **Prefect** es el "capataz": define el flujo, lo programa cada 30 minutos, reintenta las tareas que fallan y muestra estado, duración y logs en su panel (:4200).
- **Dask** es la "cuadrilla": el scheduler reparte las tareas entre **2 workers** (contenedores independientes) que las ejecutan en paralelo (panel en :8787).
- La integración es `DaskTaskRunner(address="tcp://dask-scheduler:8786")`: cada `@task` de Prefect se ejecuta en un worker de Dask y reporta su estado a Prefect.

8.2 Pipeline por ciudad

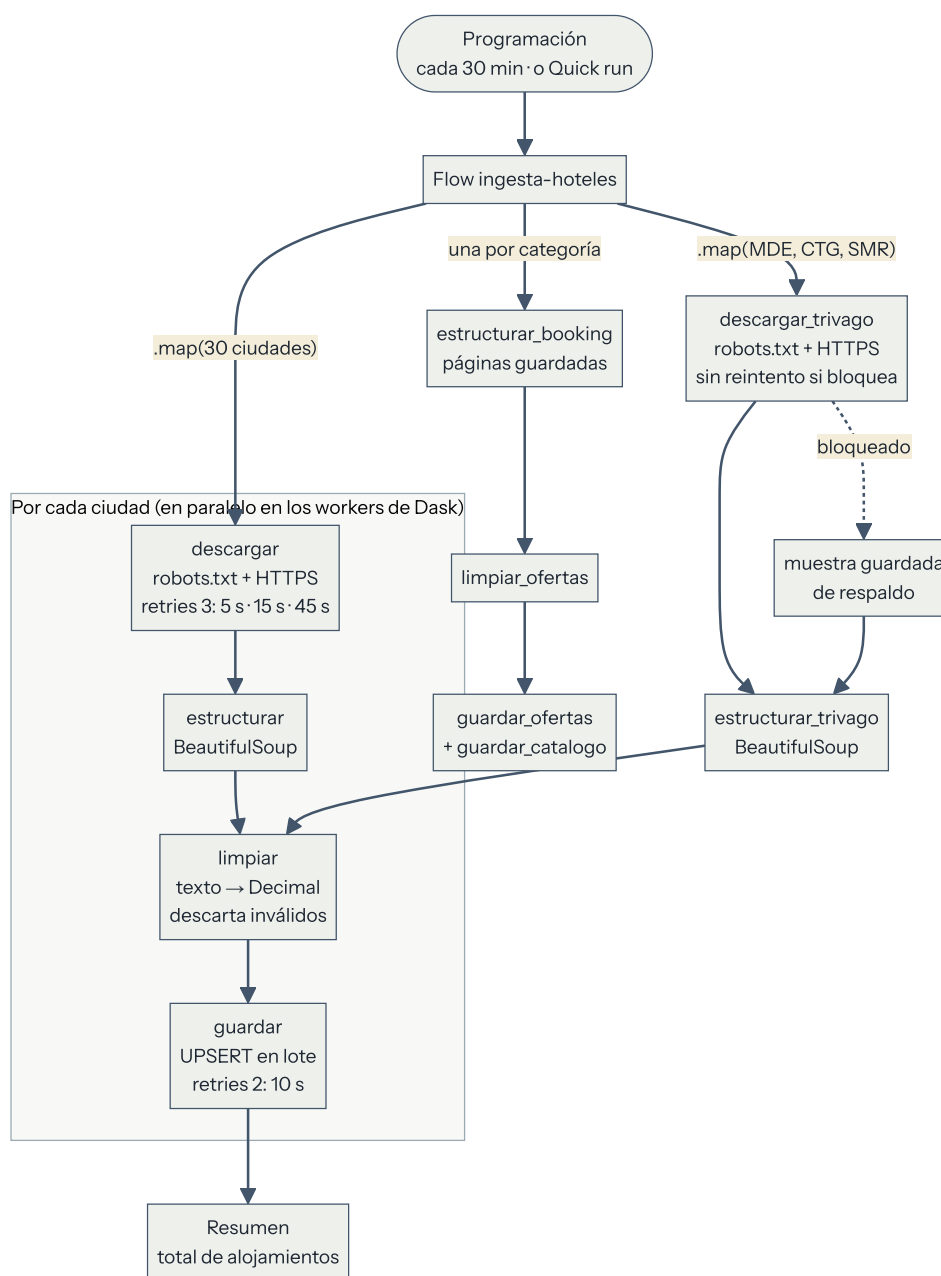


Figura 7. Pipeline por ciudad

Son **120 tareas de Hostelworld** (4 × 30 ciudades) más las de Trivago (4 × 3 ciudades) y las de Booking (estructurar, limpiar y guardar por categoría, más `guardar_catalogo` para vuelos y coches). Las tareas se encadenan con *futures*: `estructurar(MDE)` empieza apenas termina `descargar(MDE)`, sin esperar a las otras ciudades. Una ciudad que falla definitivamente no tumba a las demás; el flow solo queda `Failed` si no se pudo ingerir ninguna.

8.3 Política de reintentos

Tarea	Reintentos	Espera	Motivo
<code>descargar</code> , <code>descargar_trivago</code>	3	5 s, 15 s, 45 s (backoff)	Errores de red, DNS, 5xx del sitio; no saturar un sitio que puede estar caído. No se reintenta un bloqueo (401/403/429 o anti-bot)
<code>guardar</code>	2	10 s	Bloqueos o caídas momentáneas de la BD
<code>estructurar</code> , <code>estructurar_trivago</code> , <code>limpiar</code>	0	—	Son deterministas: reintentar daría el mismo resultado

Para la demo, `PROB_FALLO_SCRAPER=0.5` provoca fallos de red simulados y en el panel de Prefect se ve `Retry 1/3 will start 5 second(s)` seguido de `Completed`.

8.4 Fuente de datos y scraping responsable

Sitio	Resultado de la evaluación
Booking.com	Desafío anti-bot (AWS WAF), incluso en su <code>robots.txt</code> . No se descarga ni se evade: se procesan páginas guardadas a mano (§8.5b).
Kayak	<code>robots.txt</code> prohíbe <code>/flights/</code> , <code>/hotels/</code> , <code>/cars/</code> . Descartado.
Despegar	HTTP 403 (bloqueo anti-bot). Descartado.
Trivago	<code>robots.txt</code> prohíbe <code>/*/srl/</code> y <code>/hotel/</code> , pero permite las páginas de destino <code>/es-C0/odr/...</code> , que traen los hoteles en el HTML del servidor. Elegido como segunda fuente (§8.5).
Hostelworld	<code>robots.txt</code> permite <code>/hostels/...</code> y la página llega completa. Elegido .

Prácticas aplicadas: se respeta `robots.txt` con `protego` (cumple RFC 9309; `urllib.robotparser` interpreta mal reglas como `Disallow: /s?`), cacheado 30 minutos por worker (`robots_cache.py`) para no pedirlo una vez por ciudad; `User-Agent` honesto que identifica al bot; una petición por ciudad por ejecución; si aparece un desafío anti-bot, la tarea **falla** en lugar de intentar evadirlo; y un precio de Hostelworld en otra moneda (€, US\$) se descarta en vez de guardarse como si fueran pesos.

Vuelos y autos combinan los datos semilla con el catálogo que se extrae de las páginas guardadas de Booking (§8.5b).

8.5 Segunda fuente: Trivago (BeautifulSoup) con respaldo

Resultado de la prueba real (6 de octubre de 2026): Trivago respondió **HTTP 403 con desafío anti-bot** a las tres páginas de destino. Tal como está diseñado, `descargar_trivago` falla una sola vez sin reintentar (`Bloqueado`), Prefect marca las tareas siguientes como `NotReady` y el flow procesa las páginas guardadas a mano: **34** alojamientos en Medellín, **32** en Cartagena y **34** en Santa Marta. El flow termina en `Completed`: una fuente bloqueada no detiene la ingesta.

Trivago se descarga en vivo desde sus **páginas de destino** (`/es-C0/odr/hoteles-<ciudad>?search=200-<id>`). Su `robots.txt` prohíbe `/*srl/` , `/hotel/` y las imágenes de partners, pero no esas páginas, que además publica en su sitemap de destinos.

Aspecto	Decisión
Páginas	MDE <code>200-65524</code> , CTG <code>200-65529</code> , SMR <code>200-65542</code> (ids tomados del propio sitio)
Herramienta	<code>httpx</code> + BeautifulSoup. Los hoteles vienen en el HTML del servidor: no hace falta ejecutar JavaScript (Selenium)
Selectores	Atributos estables, nunca las clases CSS ofuscadas: microdatos <code>schema.org</code> (<code>itemtype=Hotel</code> , <code>itemprop</code>), el enlace <code>/oar/...?search=100-<id></code> de cada hotel (la tarjeta es el contenedor más pequeño con precio y sin otro hotel) y JSON-LD
Varias ofertas por hotel	Se guarda la más barata entre los partners
Formato de precio	<code>\$ 1.024.473</code> (punto = miles) se normaliza a dígitos antes de <code>limpiar</code>
Llave del UPSERT	<code>trivago:<id></code> : re-procesar actualiza el precio, no duplica
Descarga responsable	La misma función <code>obtener_html</code> que Hostelworld: <code>robots.txt</code> con <code>protego</code> en cada ejecución, <code>User-Agent</code> honesto, una petición por ciudad
Bloqueo del sitio	HTTP 401/403/429 o desafío anti-bot → excepción <code>Bloqueado</code> , sin reintentos (<code>retry_condition_fn</code>); un error de red sí se reintenta (5 s, 15 s, 45 s)
Respaldo	Para cada ciudad donde Trivago falló, el flow procesa la página guardada a mano en <code>ingesta/muestras/trivago/</code> (<code>fuelle = 'trivago-muestra'</code>)
HTML cambiado	Si una página llega sin hoteles reconocibles, <code>estructurar_trivago</code> falla y se ve en Prefect; <code>trivago.py --probar</code> guarda el HTML para revisar

El flow queda con dos cadenas de 4 tareas por ciudad (Hostelworld y Trivago) corriendo en paralelo en los workers de Dask, más la rama de respaldo.

8.5b Tercera fuente: ofertas de Booking.com

Booking responde con un desafío anti-bot (AWS WAF, HTTP 202) incluso a su `robots.txt` ; por RFC 9309 eso equivale a no tener permiso, así que no hay descarga automática. Las páginas de ofertas se guardan desde el navegador y el pipeline las procesa.

Aspecto	Decisión
Categorías	alojamiento (11 destinos), vuelos (50 destinos desde Bogotá), coches (197 ciudades) y atracciones (1), en <code>muestras/booking/booking-<categoria>.html</code>
Selectores	Un parser por pestaña, nunca clases CSS: alojamiento <code>data-testid="card-deal"</code> ; vuelos el estado inicial embebido (<code>window.__INITIAL_STATE__.flyAnywhere</code> , 50 destinos con IATA) y, si falta, las tarjetas <code>role="button"</code> <code>FlyAnywhere</code> ; coches <code>data-testid="in-product-interlinking-item"</code> ; atracciones el JSON de Apollo embebido (<code>AttractionsProduct</code>)
Llave del UPSERT	(<code>categoria</code> , <code>llave</code>) : <code>dest_id</code> (alojamiento), destino normalizado (vuelos), ruta del enlace (coches), slug (atracciones)

Aspecto	Decisión
Campos	destino, ciudad del sistema (código IATA), título ("Bogotá → Medellín", nombre de la atracción), detalle (país, puntos de alquiler), cantidad, precio desde y unidad
Enlace	Se guardan solo <code>dest_id</code> , <code>dest_type</code> , <code>checkin</code> y <code>checkout</code> ; se descartan <code>aid</code> , <code>label</code> y otros parámetros de rastreo
Tareas	<code>estructurar_booking</code> → <code>limpiar_ofertas</code> → <code>guardar_ofertas</code> , una cadena por categoría en Dask; independientes de los hoteles y sin tumbar el flow si fallan
Persistencia	Nuevo microservicio Ofertas con su BD <code>ofertas</code> (database per service); crea su BD y su tabla al arrancar si no existen
Catálogo	<code>guardar_catalogo</code> : los vuelos desde Bogotá y el precio medio de autos de las ciudades del sistema pasan a las BD de Vuelos y Autos (<code>fuelle = 'booking-muestra'</code>), con UPSERT por (<code>origen</code> , <code>destino</code> , <code>fuelle</code>) y (<code>ciudad</code> , <code>modelo</code>). Las ofertas no son solo vitrina: alimentan paquetes reservables
Exposición	Gateway: <code>ofertas(categoria: String): [Oferta!]</code> ; frontend: sección con pestañas; las ofertas de ciudades con paquetes enlazan a esos paquetes y las demás a Booking

8.6 Cómo se evita el cuello de botella en la persistencia

- **Una conexión y una transacción por ciudad** con `executemany`, en lugar de una conexión por fila.
- **UPSERT** (`ON CONFLICT (url) DO UPDATE`): volver a scrapear actualiza precio y rating en vez de duplicar hoteles.
- La ingesta corre **fuera del camino de las peticiones** de los clientes: los servicios de catálogo y la SAGA no esperan al scraper.

9. Ciberseguridad por diseño

Toda la seguridad de identidad y abuso se centraliza en el Gateway (`gateway/seguridad.py`) porque es la única puerta de entrada al backend.

9.1 Almacenamiento de contraseñas: Argon2id

Parámetro	Valor	Efecto
Algoritmo	Argon2id	Ganador de la Password Hashing Competition; recomendado por OWASP
<code>time_cost</code>	3	3 pasadas sobre la memoria
<code>memory_cost</code>	64 MiB	Hace carísima la fuerza bruta en GPU (poca memoria por núcleo)
<code>parallelism</code>	4	4 hilos
Sal	Aleatoria por contraseña	Mismas claves → hashes distintos

Medidas complementarias: longitud de 10 a 128 caracteres (el tope evita saturar la CPU con contraseñas gigantes); **anti-enumeración** (si el email no existe se verifica contra un hash de relleno para que el tiempo de respuesta sea igual, y el mensaje siempre es `Credenciales inválidas`); el hashing corre en un hilo aparte (`asyncio.to_thread`) para no bloquear el servidor.

9.2 Sesiones y mitigación de Session Fixation

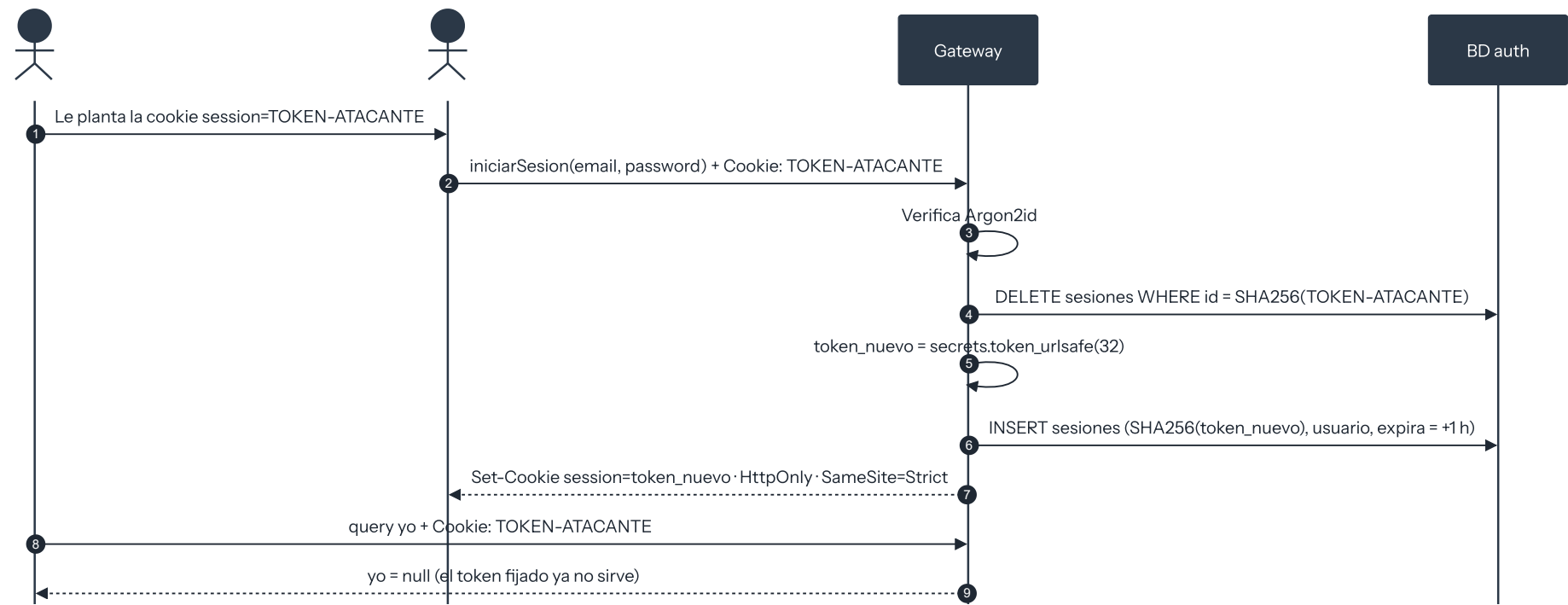


Figura 8. Sesiones y mitigación de Session Fixation

- Sesiones **del lado del servidor**: el navegador solo guarda un token aleatorio de 256 bits; en la BD se guarda su **SHA-256**, así que robar la BD no entrega sesiones utilizables.
- Cookie `HttpOnly` (un XSS no puede leerla), `SameSite=Strict` (protege contra CSRF) y `Secure` en producción (`COOKIE_SECURE=true`). Expira en 1 hora.
- `cerrarSesion` elimina la sesión en el servidor, no solo la cookie.

9.3 Rate Limiting

Como todo pasa por un único endpoint `/graphql`, el límite se aplica **por operación** dentro de cada resolver, con ventana deslizante (`limits`, *moving window*). Al excederse responde **HTTP 429**.

Operación	Límite	Protege contra
<code>iniciarSesion</code>	5/min por IP y 10 cada 15 min por email	Fuerza bruta: un atacante probando muchas cuentas, o muchas IPs atacando una cuenta
<code>registrar</code>	3/min por IP	Creación masiva de cuentas
<code>reservarPaquete</code> (checkout/pago)	10/min por IP	Abuso y denegación de servicio del checkout
<code>cancelarReserva</code>	10/min por IP	Abuso de la cancelación

IP real detrás del proxy. Con nginx delante, todas las peticiones llegarían al Gateway desde la IP de nginx y todos los usuarios compartirían un límite (uno solo podría bloquear a los demás). nginx envía `X-Real-IP`, pero esa cabecera la puede escribir cualquiera, así que el Gateway **solo le cree si la petición viene del contenedor frontend** (variable `PROXY_CONFIABLE`, resuelta por el DNS interno de Docker). Una petición directa al puerto 8000 con una `X-Real-IP` falsificada se ignora.

9.3b Control de acceso a las reservas (IDOR)

Un `sagaId` es un UUID difícil de adivinar, pero no es un secreto: se muestra en la interfaz y queda en los logs. Si bastara con conocerlo para consultar o cancelar una reserva, sería una vulnerabilidad **IDOR** (*Insecure Direct Object Reference*, OWASP A01 *Broken Access Control*). Por eso:

- `orden`, `misReservas` y `cancelarReserva` **exigen sesión**, y el `usuario_id` lo toma el Gateway de la sesión, nunca de la petición.
- Órdenes filtra **siempre** por ese `usuario_id`. La reserva de otra persona responde igual que una inexistente (`null` / "Reserva no encontrada"): ni siquiera se confirma que existe.
- El total lo calcula el servidor con los precios de cada servicio (7.3): el cliente no puede alterar lo que se cobra.

9.4 Endurecimiento del frontend

nginx añade a la aplicación: `Content-Security-Policy` restringida a `'self'` (las fuentes tipográficas se empaquetan con la app, sin CDN), `X-Frame-Options: DENY` y `frame-ancestors 'none'` (clickjacking), `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`, `server_tokens off` y un tamaño máximo de cuerpo de 64 KB. El token de sesión nunca está al alcance de JavaScript.

9.5 Seguridad de la cadena de suministro

Componente	Herramienta	Alcance	Resultado	Evidencia
6 servicios Python	<code>pip-audit</code> 2.10.1 (OSV / PyPI)	<code>requirements.txt</code> de cada servicio, versiones resueltas	0 vulnerabilidades	<code>seguridad/auditoria-dependencias.md</code>
Frontend	<code>npm audit</code> (GitHub Advisory DB)	39 paquetes de <code>package-lock.json</code> (directos y transitivos)	0 vulnerabilidades	<code>seguridad/auditoria-frontend.md</code>

La imagen del frontend se construye con `npm ci`, que instala exactamente las versiones auditadas. Las imágenes base son `slim` / `alpine` y el build del frontend es multi-etapa (la imagen final no incluye Node ni el código fuente), lo que reduce la superficie de ataque. Ambas auditorías se ejecutan en contenedores, sin instalar nada en el equipo:

```
docker run --rm -v "${PWD}:/w" -w /w python:3.12-slim sh seguridad/auditar.sh
docker run --rm -v "${PWD}:/w" -w /w node:22-alpine sh seguridad/auditar-frontend.sh
```

10. Frontend

SPA en **React 19 + Vite + Tailwind CSS 4**, compilada a archivos estáticos y servida por **nginx**. Tiene dos vistas sobre el mismo código:

- **Vista de cliente** (por defecto): solo lo que un viajero necesita. Sin paneles técnicos ni nombres internos.
- **Modo demo** (`http://localhost:3000/?demo`): añade las herramientas de la sustentación (panel técnico de la SAGA, Inspector GraphQL, enlaces a Prefect, Dask y GraphiQL, y la etiqueta de origen de cada precio).

Elemento	Vista	Qué hace	Requisito que evidencia
Buscador	Cliente	"Selecciona tu destino" : todos los destinos de la página (con paquete y solo con ofertas de Booking), con buscador por ciudad o país. "Búsqueda por paquetes" : un botón por destino con paquete (<code>query destinos</code>). Noches y personas (1 a 9) → <code>query paquetes</code> , o las ofertas de Booking del destino si no tiene paquete	Consumo de GraphQL desde el frontend
Responsive	Cliente	Probado en 360, 390, 768, 1024 y 1366 px sin desplazamiento horizontal: carrusel de ofertas y fila deslizable de destinos en celular, selector a pantalla completa, tarjeta de paquete reorganizada, desplazamiento a los resultados al buscar	Usabilidad en cualquier dispositivo
Tarjetas de paquete	Cliente	Vuelo × personas, hotel × habitaciones, auto × autos, total y precio por persona; proveedor del precio (Hostelworld o Trivago)	Datos reales ingeridos por Dask/Prefect
Pantalla de acceso	Cliente	Lo primero que se ve sin sesión: iniciar sesión o crear cuenta (validación en vivo, confirmación de contraseña). Nada de la app se muestra antes de entrar	Argon2id, sesiones, rate limiting
Tu reserva	Cliente	Resultado en lenguaje de cliente: confirmada (con código <code>WS-...</code> y resumen), no completada (todo se canceló solo) o en revisión	Resultado de la SAGA
Mis reservas	Cliente	Reservas del usuario guardadas en el servidor (<code>query misReservas</code>): destino, hotel, noches, personas, total y estado. Actualizar estado (<code>query orden</code>) y Cancelar reserva con confirmación (<code>mutation cancelarReserva</code>)	Bitácora de la SAGA, compensación y control de acceso
Panel <i>Transacción SAGA</i>	Demo	Pasos vuelo → hotel → auto y compensaciones en orden inverso	Demostración de la SAGA
Inspector GraphQL	Demo	Cada operación enviada: consulta exacta, HTTP, bytes y tiempo	Sin over-fetching; un único endpoint
Enlaces a Prefect, Dask y GraphiQL	Demo	Accesos directos	Observabilidad

Decisiones: cliente GraphQL propio sobre `fetch` (sin Apollo: bundle pequeño y cada petición es visible en el inspector); la sesión depende solo de la cookie `HttpOnly`; si el usuario intenta reservar sin sesión, se le pide iniciar sesión y la reserva se retoma automáticamente; en móvil, el panel de la SAGA se desplaza a la vista al reservar. Paleta tomada del paisaje del cañón de Waimea (verdes, oliva, mostaza y azules pizarra).

11. Despliegue con Docker Compose

```
docker compose up --build      # levanta todo
docker compose down -v        # apaga y borra la BD (necesario si cambia db/init.sql)
```

Servicio	Imagen / build	Puerto publicado	Depende de
db	postgres:16	—	— (healthcheck <code>pg_isready</code>)
vuelos, hoteles, autos	./<servicio> (Python 3.12-slim)	—	db sano
ofertas	./ofertas	—	db sano
ordenes	./ordenes	—	db sano, participantes
gateway	./gateway	8000 (GraphQL)	db sano, los 5 servicios
frontend	./frontend (Node → nginx 1.27-alpine)	3000	gateway
tunnel (perfil publico)	cloudflare/cloudflared	— (URL pública https)	frontend
prefect-server	./ingesta	4200	— (healthcheck <code>/api/health</code>)
dask-scheduler	./ingesta	8787	—
dask-worker x2	./ingesta (replicas: 2)	—	scheduler, db sano
ingesta	./ingesta (python flow.py)	—	prefect-server sano, workers

Detalles relevantes: `depends_on` con `condition: service_healthy` evita que los servicios arranquen antes que Postgres; los cuatro contenedores de ingesta comparten **la misma imagen** para garantizar versiones idénticas de Dask y Prefect (si scheduler y workers difieren, Dask falla al serializar); las credenciales y URLs llegan por variables de entorno.

11.1 URL pública (Cloudflare Tunnel)

Para la sustentación la app se publica con **Cloudflare Tunnel** (`docker compose --profile publico up -d`): el contenedor `tunnel` abre una conexión saliente hacia Cloudflare y recibe una URL `https://...trycloudflare.com`. No se abren puertos en el router ni se crean cuentas.

Aspecto	Decisión
Qué se publica	Solo el frontend (nginx). Prefect, Dask, la BD y los microservicios siguen en la red interna
HTTPS	Lo termina Cloudflare; nginx y el Gateway hablan HTTP dentro de Docker
IP real del visitante	nginx toma <code>CF-Connecting-IP</code> solo de IPs privadas (el contenedor del túnel) y la pasa como <code>X-Real-IP</code> ; el Gateway solo le cree a esa cabecera si viene del contenedor <code>frontend</code>

Aspecto	Decisión
Cookie de sesión	El Gateway la marca <code>Secure</code> cuando <code>X-Forwarded-Proto: https</code> llega desde el proxy de confianza
Limitación	La URL existe mientras el PC y Docker estén encendidos y cambia en cada reinicio. Un despliegue permanente iría en una VM (por ejemplo EC2) con dominio propio

12. Guion de la demostración

Punto exigido	Pasos
(a) Prefect monitoreando los flows	<code>http://localhost:4200</code> → <i>Deployments</i> → <code>ingesta-hoteles / hostelworld-cada-30-min</code> → <i>Quick run</i> . Mostrar las tareas (4 por destino), sus estados, duración y logs (<code>MDE: N</code> <code>hostales guardados...</code>). Con <code>PROB_FALL0_SCRAPER=0.5</code> se ven los reintentos.
(b) Tareas distribuidas en Dask	Durante el <i>Quick run</i> , abrir <code>http://localhost:8787</code> : las tareas se reparten entre los 2 workers (<i>Task Stream</i> , <i>Workers</i>).
(c) Consumo de GraphQL desde el frontend	<code>http://localhost:3000/?demo</code> : buscar Medellín para 4 personas, mostrar las tarjetas ("Hostelworld · en vivo", "Trivago · muestra") y abrir el Inspector GraphQL para ver la consulta exacta y el tamaño de la respuesta. Mostrar también <code>http://localhost:3000</code> (vista de cliente, sin paneles técnicos).
(d) Fallo transaccional y compensaciones	Crear <code>.env</code> con <code>FORZAR_FALL0_AUTOS=true</code> → <code>docker compose up -d autos</code> → reservar desde <code>http://localhost:3000/?demo</code> → el cliente ve "No pudimos completar tu viaje" y el panel técnico muestra <i>Compensada</i> con las cancelaciones en orden inverso → <code>docker compose logs ordenes</code> → <code>docker compose exec db psql -U postgres -d hoteles -c "SELECT * FROM reservas"</code> (estado <code>CANCELADA</code>).
Seguridad (extra)	Hash <code>\$argon2id\$...</code> en la BD <code>auth</code> ; Session Fixation con <code>curl</code> (README §6c); 6 logins fallidos → HTTP 429; reportes de <code>pip-audit</code> y <code>npm audit</code> .

13. Limitaciones conocidas y trabajo futuro

Limitación	Impacto	Mejora propuesta
El orquestador no retoma SAGAs interrumpidas. Si Órdenes se cae a mitad de una SAGA, la fila queda en <code>EN_CURSO</code> .	Posibles reservas parciales hasta una revisión manual. La bitácora sí registra dónde iba.	Un proceso de recuperación al arrancar que compense las SAGAs <code>EN_CURSO</code> antiguas (patrón <i>Saga Log + recovery</i>).
La SAGA es síncrona (la mutation espera el resultado).	Si hay compensaciones con reintentos, la respuesta puede tardar segundos.	Responder <code>EN_CURSO</code> de inmediato y que el frontend consulte <code>orden(sagaId)</code> o use una <i>subscription</i> .
Rate limiter en memoria.	Válido para una sola réplica del Gateway.	<code>RedisStorage</code> compartido entre réplicas.
Precios como <code>Float</code> en GraphQL. Se almacenan y limpian como <code>NUMERIC</code> / <code>Decimal</code> , pero viajan como <code>float</code> en JSON y el total se suma en <code>float</code> .	Posibles errores de redondeo en centavos.	Escalar <code>Decimal</code> en Strawberry (serializado como texto) y sumar con <code>Decimal</code> .
Producto cartesiano de paquetes. Se combinan todos los vuelos × hoteles × autos de la ciudad.	Con muchos datos, la respuesta crece rápido.	Paginación (<code>first</code> / <code>after</code>) y filtros en la query.
Sin reserva de inventario ni pago real.	Las reservas no descuentan disponibilidad.	Paso de pago como participante adicional de la SAGA y control de cupos.
Sin fechas de viaje. Se reserva por número de noches, sin fecha de ida.	No hay política de cancelación por fecha (se puede cancelar cualquier reserva confirmada).	Fechas de ida y vuelta en la búsqueda y una ventana de cancelación gratuita.
Los autos de Booking son un precio medio por ciudad , no un modelo concreto.	El cliente ve "Auto estándar · precio medio en Booking".	Una fuente de alquiler de autos con modelos y disponibilidad.
Credenciales de desarrollo en <code>docker-compose.yml</code> (<code>POSTGRES_PASSWORD: dev</code>) y HTTP sin TLS en local.	Aceptable solo en desarrollo.	Docker secrets o un gestor de secretos; TLS y <code>COOKIE_SECURE=true</code> en producción.
El scraper depende del HTML del sitio.	Si Hostelworld cambia sus clases CSS, se guardan 0 hostales (el flow lo reporta como fallo).	Pruebas de contrato sobre HTML de ejemplo y alertas desde Prefect.
Trivago puede bloquear al bot o cambiar su HTML.	Sin datos en vivo de Trivago para esa ciudad.	El respaldo con páginas guardadas cubre la demo; a largo plazo, la API de afiliados de Trivago.
Sin pruebas automatizadas.	Las verificaciones son manuales.	Pruebas de la SAGA (happy path y compensación) con <code>pytest</code> + contenedores efímeros.

14. Conclusiones

- El problema de las **reservas huérfanas** se resuelve con una **SAGA orquestada**: cada transacción termina confirmada por completo o compensada por completo, y si una compensación no se logra queda marcada en `REQUIERE_ATENCION` en vez de fallar en silencio. La idempotencia por `saga_id` es lo que hace seguro reintentar.
- *Database per service* da autonomía a cada microservicio y es la razón de ser de la SAGA: sin transacciones globales, la consistencia se construye con acciones y compensaciones.
- El **Gateway GraphQL** unifica el acceso, permite al cliente pedir solo lo que necesita y concentra la seguridad en un único punto de entrada.
- **Dask + Prefect** sacan la sincronización de tarifas del camino de las peticiones: el scraping se paraleliza en workers, se reintenta ante fallos de red, se persiste en lote y es observable en tiempo real.
- La seguridad se integró desde el diseño: Argon2id, sesiones regeneradas en cada login, rate limiting por operación con IP real verificada, cabeceras de seguridad en el frontend y auditorías de dependencias sin vulnerabilidades.